

FLUX-A-FLOW

Direct Farm-to-Buyer Marketplace Platform

Software Requirements Specification (SRS)

Document Title	Software Requirements Specification
Project Name	Flux-A-Flow
Version	1.0
Date	June 2026
Repository	github.com/richardcreatech/Flux-A-Flow

STUDENT NAMES	STUDENT ID	ROLE
Sanni AbdurRahman Omofolarin	20230957	Requirement Specification
Muhammad Saleh Salim	20230985	Frontend Development
David Gbemudu	211212148	Designer
Mgbahurike Richard Uzoma	20230952	Backend Developer

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Overview
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.6 Development Methodology
- 3. Specific Requirements
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.7 Object Oriented Models

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed understanding of the direct farm-to-market sales platform.

This will define parts of the system such as the app's core features, user interfaces as well as a comprehensive description of the functional and non-functional requirements for direct farm-to-market.

This document serves as the primary point of reference for developers, designers, and project stakeholders involved in the building and constant refinements.

It outlines the system's objectives and constraints to ensure alignment throughout the development lifecycle.

This Software Requirements Specification (SRS) document provides a comprehensive description of the Flux-A-Flow platform, a direct farm-to-buyer marketplace that connects agricultural producers with consumers, eliminating traditional intermediaries. The purpose of this document is to:

- Define the functional and non-functional requirements of the system.
- Serve as a reference for developers, testers, project managers, and stakeholders.
- Establish a shared understanding of what the system is expected to do and the constraints under which it must operate.
- Provide a baseline against which the final product will be evaluated.

This document is intended for software engineers, UI/UX designers, system architects, quality assurance teams, and academic reviewers associated with the Flux-A-Flow project.

1.2 Scope

In traditional agricultural markets, farmers rarely sell their produce directly to consumers or retailers. Instead, intermediaries known as middlemen purchase products from farmers in bulk and resell them to buyers, retailers, or marketplaces. This process forms the conventional farm-to-market supply chain used in many communities.

Farmers frequently rely on middlemen to handle transportation, distribution, market access, and negotiations with buyers. This support enables farmers to reach larger markets that might otherwise be inaccessible due to distance, limited resources, or limited means of connecting with buyers. This reduces the burden on farmers who may not have the means to market or transport their products independently.

Despite their advantages, farmers may receive lower profits because products are often purchased at reduced prices and resold at significantly higher rates. In many cases, farmers also lack transparency regarding current market prices and buyer demand, making them heavily dependent on these middlemen.

This dependency can lead to limited direct communication between farmers and buyers.

Middlemen play an important role in assisting farmers with product distribution, transportation, logistics, and access to larger markets. Their involvement helps simplify the selling process for farmers who may lack the resources or connections needed to reach buyers directly. However, this process can reduce the profit earned by farmers, as middlemen often purchase products at lower prices and resell them at higher rates. This creates a dependency on intermediaries and limits direct interaction between farmers and buyers.

Flux-A-Flow is a full-stack web application that establishes a direct marketplace between farmers (sellers) and end consumers or buyers. The platform aims to:

- Provide farmers with a dedicated digital storefront to list, manage, and sell their produce.
- Allow buyers to browse, search, order, and pay for fresh farm produce online.
- Facilitate transparent pricing, reduce dependency on middlemen, and increase farmers' profit margins.
- Support real-time order tracking, messaging between buyers and sellers, and user account management.

The system is built using a JavaScript-based technology stack comprising Node.js on the backend, an EJS/ReactJS/CSS frontend, and a NoSQL database. The application is served as a web platform accessible via standard browsers on desktop and mobile devices.

Out of scope for this version are: native mobile applications (iOS/Android), multi-currency international payments, and third-party logistics integration.

1.3 Overview

This SRS is organized as follows:

Flux-A-Flow provides farmers with a method by which they can sell their farm produce to other farmers, wholesalers, restaurants and retailers through a digital platform thus eliminating a traditional middleman.

- Section 1 – Introduction: Provides the purpose, scope, and structural overview of this document.
- Section 2 – Overall Description: Describes the product perspective, key functions, user classes, and development methodology.
- Section 3 – Specific Requirements: Details the functional requirements, non-functional requirements, and object-oriented models of the system.

Together, these sections form a complete specification that guides the design, development, and testing of Flux-A-Flow.

2. Overall Description

2.1 Product Perspective

Flux-A-Flow operates as a standalone web platform that introduces a direct digital channel between agricultural producers and buyers. Traditional agricultural supply chains involve multiple layers of middlemen—distributors, wholesalers, and retailers—each adding cost and reducing freshness by the time produce reaches the consumer.

Flux-A-Flow positions itself as a digital farmer's market by providing:

- A web-accessible storefront that farmers self-manage without technical expertise.
- A buyer-facing marketplace with search, filter, and product-discovery capabilities.
- A backend system that processes orders, authenticates users, and maintains transactional integrity.

The application interacts with the following external entities:

Entity	Interaction Type	Description
Web Browser	Client Interface	Users access Flux-A-Flow via modern web browsers (Chrome, Firefox, Safari, Edge)
Database Server	Data Persistence	Stores user profiles, product listings, orders, and transactional records
Payment Gateway (Future)	External API	Handles secure payment processing for buyer orders
Email Service	Notification	Sends confirmation emails, password resets, and order updates
File Storage	Media Upload	Stores product images uploaded by farmers

The system is deployed as a web server (Node.js/Express) that serves dynamically rendered HTML pages via EJS templates. The frontend communicates with backend routes via standard HTTP requests, making it a server-rendered multi-page application (MPA) rather than a Single Page Application (SPA).

2.2 Product Functions

As a farmer, I should be able to create an account, log into the platform, upload farm products, set product prices, adjust product quantities, manage uploaded products, view incoming orders, and communicate directly with potential buyers through the platform.

As a buyer, I should be able to create an account, log into the platform, browse available farm products, search for specific products, view product details, place orders, track orders, and communicate directly with farmers before or after purchases.

As an administrator, I should be able to manage users on the platform, monitor uploaded products, manage orders, remove invalid or inappropriate listings, monitor activities within the system, and maintain the overall operation and security of the platform.

Flux-A-Flow delivers the following core functional capabilities:

Farmer (Seller) Functions

- Register and create a verified farmer profile.
- Add, update, and remove product listings with names, descriptions, prices, quantities, and images.
- View and manage incoming orders (accept, fulfill, cancel).
- View sales reports and revenue summaries.
- Communicate with buyers via in-platform messaging.

Buyer (Consumer) Functions

- Register and maintain a personal buyer account.
- Browse and search the product catalog with filters (category, price range, location, availability).
- Add products to a shopping cart and place orders.
- Track order status in real-time.
- Leave reviews and ratings for products and farmers.
- Manage saved addresses and payment preferences.

Platform Functions

- User authentication and authorisation (role-based access control).
- Secure session management using cookies/tokens.
- Email notifications for account actions and order events.
- Responsive landing page and marketing content.

2.6 Development Methodology

Flux-A-Flow is developed using an Agile software development methodology, specifically a lightweight Scrum-inspired approach adapted for a small team setting. This methodology was selected for its iterative nature, which allows the team to incorporate feedback and evolve requirements as the project progresses.

Key aspects of the development methodology include:

- **The application is built in short development cycles (sprints), with working software delivered at the end of each iteration.** Iterative Development:
- **Each sprint targets a specific module or feature set — e.g., user authentication, product listings, order management — allowing focused development and testing.** Feature-Driven Increments:
- **All source code is managed in the public GitHub repository (richardcreatech/Flux-A-Flow), with branches used for feature development and pull requests for integration.** Version Control via Git/GitHub:
- **The backend follows the Model-View-Controller (MVC) design pattern, with clear separation between data models, business logic routes, and EJS-templated views.** MVC Architecture:
- **Code reviews and issue tracking are conducted via GitHub Issues and pull requests, ensuring quality and knowledge sharing among contributors.** Collaborative Review:

The technology stack was selected to maximize development speed with minimal configuration overhead:

Layer	Technology	Role
Backend Runtime	Node.js	Server-side JavaScript execution environment
Web Framework	Express.js	HTTP routing, middleware, and REST API handling
Template Engine	EJS	Server-side HTML rendering with dynamic data
Frontend	ReactJS / CSS3	Structure and styling of UI pages
Scripting	JavaScript (ES6+)	Client-side interactivity and DOM manipulation
Database	MongoDB	Persistent data storage
Version Control	Git / GitHub	Source code management and collaboration

3. Specific Requirements

3.1 Functional Requirements

The functional requirements below describe what the Flux-A-Flow system shall do. Each requirement is identified by a unique ID for traceability.

3.1.1 User Account Management

ID	Requirement	Priority
FR-001	The system shall allow new users to register with a full name, email address, password, and role (Farmer or Buyer).	High
FR-002	The system shall validate that email addresses are unique during registration.	High
FR-003	The system shall hash user passwords using a secure algorithm (e.g., bcrypt) before storing them.	High
FR-004	The system shall allow registered users to log in with their email and password.	High
FR-005	The system shall maintain user sessions using secure HTTP cookies or JWT tokens.	High
FR-006	The system shall allow users to log out, invalidating their current session.	Medium
FR-007	The system shall allow users to reset their password via an email-based reset link.	Medium
FR-008	The system shall display a user profile page with editable fields (name, contact, address, profile picture).	Medium

3.1.2 Product Listing Management (Farmer)

ID	Requirement	Priority
FR-009	The system shall allow authenticated Farmers to create new product listings with a name, description, category, price, quantity, and at least one image.	High
FR-010	The system shall allow Farmers to update any field of their existing product listings.	High
FR-011	The system shall allow Farmers to delete their product listings.	High
FR-012	The system shall allow Farmers to mark a product as out of stock or unavailable without deleting it.	Medium

ID	Requirement	Priority
FR-013	The system shall display each Farmer's full product inventory on their dashboard.	High
FR-014	The system shall support image upload for product listings with a maximum file size of 5MB.	Medium

3.1.3 Product Browsing and Discovery (Buyer)

ID	Requirement	Priority
FR-015	The system shall display a public product catalog accessible to unauthenticated users.	High
FR-016	The system shall allow buyers to search products by keyword.	High
FR-017	The system shall allow buyers to filter products by category, price range, and availability.	High
FR-018	The system shall display individual product detail pages with image, description, price, seller info, and reviews.	High
FR-019	The system shall recommend related products on the product detail page.	Low
FR-020	The system shall display featured or promoted listings on the homepage.	Medium

3.1.4 Shopping Cart and Order Management

ID	Requirement	Priority
FR-021	The system shall allow authenticated Buyers to add products to a shopping cart.	High
FR-022	The system shall allow Buyers to update product quantities or remove items from the cart.	High
FR-023	The system shall calculate and display an order subtotal, inclusive of applicable delivery fees.	High
FR-024	The system shall allow Buyers to place an order from the cart, selecting a delivery address.	High
FR-025	The system shall notify the Farmer via email when a new order is placed.	Medium
FR-026	The system shall allow Farmers to update order status (Pending, Confirmed, Shipped, Delivered, Cancelled).	High

ID	Requirement	Priority
FR-027	The system shall display order history for both Buyers (purchases) and Farmers (sales).	High
FR-028	The system shall allow Buyers to cancel an order if its status is still Pending.	Medium

3.1.5 Reviews and Ratings

ID	Requirement	Priority
FR-029	The system shall allow Buyers to leave a star rating (1–5) and text review for a product after delivery.	Medium
FR-030	The system shall display the average rating and all reviews on the product detail page.	Medium
FR-031	The system shall prevent a Buyer from reviewing the same product more than once.	Low

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the Flux-A-Flow system.

3.2.1 Performance

ID	Requirement
NFR-001	The system shall load any product listing or category page within 3 seconds under normal network conditions (broadband).
NFR-002	The system shall support up to 500 concurrent users without significant degradation in response time.
NFR-003	The system shall complete database queries for product searches in under 1 second for datasets up to 10,000 listings.

3.2.2 Security

ID	Requirement
NFR-004	All user passwords shall be hashed using bcrypt with a salt factor of at least 10.
NFR-005	The system shall enforce HTTPS for all pages and API endpoints in production.
NFR-006	The system shall protect all authenticated routes against unauthorised access through session validation.
NFR-007	The system shall sanitise all user inputs to prevent Cross-Site Scripting (XSS) and SQL/NoSQL injection attacks.
NFR-008	The system shall implement CSRF protection on all state-changing form submissions.

3.2.3 Usability

ID	Requirement
NFR-009	The user interface shall be fully responsive and functional on screen widths ranging from 320px (mobile) to 1920px (desktop).
NFR-010	The system shall provide informative error messages for all failed operations (e.g., login failure, form validation errors).
NFR-011	A new user with no prior experience shall be able to complete a product purchase within 5 minutes of registration.
NFR-012	The system shall provide a consistent navigation structure across all pages.

3.2.4 Reliability and Availability

ID	Requirement
NFR-013	The system shall achieve an uptime of at least 99% during operational hours.
NFR-014	The system shall implement data backup mechanisms to prevent data loss in the event of a server failure.
NFR-015	The system shall gracefully handle server-side errors, displaying user-friendly error pages rather than raw stack traces.

3.2.5 Maintainability

ID	Requirement
NFR-016	The codebase shall follow the MVC architectural pattern to ensure separation of concerns.
NFR-017	All modules, routes, and components shall include inline comments explaining their purpose.
NFR-018	The project shall include a README documenting setup instructions, dependencies, and deployment steps.
NFR-019	The system shall use environment variables (.env) to store configuration secrets, never hardcoded values.

3.7 Object-Oriented Models

This section describes the key object-oriented models that underpin the Flux-A-Flow system. These models capture the real-world entities, their attributes, behaviours, and relationships.





